

get-mempool subscription lifecycle specification.

The draft (“get-mempool”) makes the *mempool* stream the one open-ended request stream: the responder streams a snapshot and then updates until the stream or connection ends; a requester that no longer wants the subscription cancels the responder’s sending direction with *CANCELLED*. And: “A node MUST NOT open more than one concurrent *get – mempool stream to the same peer*; a second concurrent subscription is a connection error of type *PROTOCOL_ERROR*”

The cancel and a re-subscription travel on different streams, and streams are mutually independent (“Transport Requirements”), so the responder can observe the new subscription’s type byte before the old subscription’s cancel. At that moment it is serving one stream and sees a second | from a requester that, from its own point of view, never had two subscriptions at once. This is the announcement-stream replacement race (see protocol.tla, Finding 1) recurring in a second rule of the draft.

The StrictSub constant selects the reading:

TRUE	a second subscription observed while one is being served is a connection error (the literal text). NoHonestProtocolError is violated.
FALSE	the responder treats the old subscription as cancelled (its cancel is on the way), ends it, and serves the new one. All properties hold.

Zebra’s draft implementation is strict: ‘mempool_subscribed.swap(true)’ fails the connection on a second subscription, and the flag is cleared only when serving ends | its conformance notes call the mitigation “prompt cancel detection”, i.e. timing. Its own requester keeps one subscription per connection and never re-subscribes after a cancel, so Zebra-to-Zebra does not reach the race; the draft explicitly anticipates cancel/re-open churn from other requesters (it recommends rate-limiting it, which is a FLOOD-coded answer to a liveness problem, not an ordering rule).

The responder-initiated end (finishing the stream on shutdown or snapshot lag) is not modeled: when the responder ends the stream itself its serving slot is already free before the requester can react, so that direction has no race.

See ../documents/v2-modeling.md for the full write-up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT *MaxSubs* *subscription attempts available to the requester*
 CONSTANT *StrictSub* *see module comment*

Slots $\triangleq$ 1 .. *MaxSubs*

One record per subscription attempt *k* :

r requester *s* view: *none* | *open* | *cancelled* | *closed*
s responder’s view: *none* (not yet observed) | *serving* | *done*
open_if the stream’s type byte is in flight toward the responder
stop_if the requester’s *CANCELLED* stop-sending is in flight
end_if the responder’s reset/finish is in flight toward the requester

VARIABLES *subs*, *closed*

$vars \triangleq \langle subs, closed \rangle$

$NoCode \triangleq \text{"none"}$

$NullSub \triangleq [r \mapsto \text{"none"}, s \mapsto \text{"none"},$
 $open_if \mapsto \text{FALSE}, stop_if \mapsto \text{FALSE}, end_if \mapsto \text{FALSE}]$

$FreeSlots \triangleq \{k \in Slots : subs[k].r = \text{"none"}\}$

$Init \triangleq$

$\wedge subs = [k \in Slots \mapsto NullSub]$
 $\wedge closed = NoCode$

The requester opens a get-mempool stream when it considers itself unsubscribed ("MUST NOT open more than one concurrent stream", judged from its own state — it cannot judge from the responder's).

$Subscribe \triangleq$

$\wedge closed = NoCode$
 $\wedge \forall k \in Slots : subs[k].r \neq \text{"open"}$
 $\wedge FreeSlots \neq \{\}$
 $\wedge LET k \triangleq CHOOSE k \in FreeSlots : \forall j \in FreeSlots : k \leq j$
 $IN \quad subs' = [subs \text{ EXCEPT } ![k].r = \text{"open"}, ![k].open_if = \text{TRUE}]$
 $\wedge UNCHANGED closed$

The requester cancels its live subscription with *CANCELLED*, intending to re-subscribe (the churn the draft anticipates). Bounded by the free slots so the model quiesces with a live subscription.

$Cancel \triangleq$

$\exists k \in Slots :$
 $\wedge closed = NoCode$
 $\wedge subs[k].r = \text{"open"}$
 $\wedge FreeSlots \neq \{\}$
 $\wedge subs' = [subs \text{ EXCEPT } ![k].r = \text{"cancelled"}, ![k].stop_if = \text{TRUE}]$
 $\wedge UNCHANGED closed$

The responder observes attempt *k*'s type byte. If it is still serving an attempt whose cancel it has not yet seen, this looks like a second concurrent subscription:

StrictSub $\rightarrow$ connection error *PROTOCOL_ERROR* (the literal text)
otherwise $\rightarrow$ the stream the requester opened LATER wins: a newer stream supersedes the one being served; an older one is the stale open of an already-cancelled subscription, observed late, and is refused without penalty.

$$\begin{array}{l}
SeeOpen \triangleq \\
\quad \exists k \in Slots : \\
\quad LET \quad dup \triangleq \exists j \in Slots : j \neq k \wedge subs[j].s = "serving" \\
\quad \quad newer \triangleq \forall j \in Slots : subs[j].s = "serving" \Rightarrow k > j \\
IN \\
\quad \wedge closed = NoCode \\
\quad \wedge subs[k].open_if \\
\quad \wedge \vee \wedge StrictSub \wedge dup \\
\quad \quad \wedge closed' = "PROTOCOL_ERROR" \\
\quad \quad \wedge subs' = [j \in Slots \mapsto NullSub] \\
\quad \vee \wedge \neg StrictSub \wedge dup \wedge \neg newer \\
\quad \quad \wedge closed' = closed \\
\quad \quad \wedge subs' = [subs \text{ EXCEPT } ![k].s = "done", ![k].open_if = FALSE, \\
\quad \quad \quad \quad \quad \quad \quad \quad ![k].end_if = TRUE] \\
\quad \vee \wedge \neg (StrictSub \wedge dup) \wedge (newer \vee \neg dup) \\
\quad \quad \wedge closed' = closed \\
\quad \quad \wedge subs' = [j \in Slots \mapsto \\
\quad \quad \quad IF \quad j = k \\
\quad \quad \quad THEN \quad [subs[k] \text{ EXCEPT } !.s = "serving", !.open_if = FALSE] \\
\quad \quad \quad ELSE \quad IF \quad subs[j].s = "serving" \\
\quad \quad \quad THEN \quad [subs[j] \text{ EXCEPT } !.s = "done", !.end_if = TRUE] \\
\quad \quad \quad ELSE \quad subs[j]]
\end{array}$$
$$\begin{aligned}
\text{SeeStop} &\triangleq \\
&\exists k \in \text{Slots} : \\
&\quad \wedge \text{closed} = \text{NoCode} \\
&\quad \wedge \text{subs}[k].\text{stop_if} \\
&\quad \wedge \text{subs}' = [\text{subs} \text{ EXCEPT} \\
&\quad \quad ! [k].\text{stop_if} = \text{FALSE}, \\
&\quad \quad ! [k].\text{open_if} = \text{FALSE}, \\
&\quad \quad ! [k].s = \text{IF } @ = \text{"serving"} \text{ THEN "done" ELSE } @, \\
&\quad \quad ! [k].\text{end_if} = @ \vee \text{subs}[k].s = \text{"serving"}]
\end{aligned}$$

$\wedge \text{UNCHANGED } closed$

The requester observes the responder's reset/finish of attempt k .

$RecvEnd \triangleq$

$\exists k \in Slots :$

$\wedge closed = NoCode$

$\wedge subs[k].end_if$

$\wedge subs' = [subs \text{ EXCEPT } ![k].end_if = FALSE,$

$![k].r = \text{IF } @ \in \{ \text{"open"}, \text{"cancelled"} \} \text{ THEN "closed" ELSE } @]$

$\wedge \text{UNCHANGED } closed$

$Next \triangleq Subscribe \vee Cancel \vee SeeOpen \vee SeeStop \vee RecvEnd$

Per-slot fairness on the responder's and requester's receive actions (the existential form can be discharged by other slots while one starves — see *dial.tla*), plus fairness on subscribing. Cancel is never forced.

$Spec \triangleq$

$Init$

$\wedge \Box [Next]_{vars}$

$\wedge WF_{vars}(Subscribe)$

$\wedge \forall k \in Slots :$

$\wedge WF_{vars}(closed = NoCode \wedge subs[k].open_if \wedge SeeOpen)$

$\wedge WF_{vars}(closed = NoCode \wedge subs[k].stop_if \wedge SeeStop)$

$\wedge WF_{vars}(closed = NoCode \wedge subs[k].end_if \wedge RecvEnd)$

Liveness: the requester ends up subscribed and served.

$EventuallySubscribed \triangleq$

$\Diamond \Box \exists k \in Slots : subs[k].r = \text{"open"} \wedge subs[k].s = \text{"serving"}$

Safety invariants.

$TypeOK \triangleq$

$\wedge subs \in [Slots \rightarrow [r : \{ \text{"none"}, \text{"open"}, \text{"cancelled"}, \text{"closed"} \},$

$s : \{ \text{"none"}, \text{"serving"}, \text{"done"} \},$

$open_if : \text{BOOLEAN}, stop_if : \text{BOOLEAN}, end_if : \text{BOOLEAN}]]$

$\wedge closed \in \{ NoCode, \text{"PROTOCOL_ERROR"} \}$

The requester's MUST: it never considers two subscriptions open at once.

$RequesterSingleton \triangleq$

$Cardinality(\{k \in Slots : subs[k].r = \text{"open"}\}) \leq 1$

The responder never serves two streams at once, in either reading.

$ResponderSingleton \triangleq$

$$Cardinality(\{k \in Slots : subs[k].s = \text{"serving"}\}) \leq 1$$

No interleaving of conformant behaviour ends in a protocol error.
 $NoHonestProtocolError \stackrel{\Delta}{=} closed = NoCode$
